

Controllo della Qualità del Servizio (QoS HTB) e Sicurezza tramite Dynamic ARP Inspection in Reti SDN basate su OpenFlow 1.3

RELAZIONE TECNICA DI PROJECT WORK

Studenti Referenti:

- Antonio Auricchio Matr. DE9000022
- Salvatore Marrandino Matr. DE9000118
- Alessandro Ortoli Matr. DE9000038

Repository e Consegna:

Repo Git: [giovannistanco/NCIS_unina](https://github.com/giovannistanco/NCIS_unina) (Branch 2026)

Cartella PR: Auricchio_Marrandino_Ortoli

1 Introduzione e Obiettivi

Le architetture **Software-Defined Networking (SDN)** disaccoppiano formalmente il piano di controllo (*control plane*) dal piano dati (*data plane*), abilitando una logica di forwarding centralizzata, programmabile e flessibile. Questo project work affronta due problematiche critiche all'interno di infrastrutture SDN multi-subnet:

1. **Quality of Service (QoS):** Garantire la corretta allocazione di banda e l'isolamento dei flussi mission-critical rispetto a flussi concorrenti best-effort congestionanti su un canale condiviso (*bottleneck*).
2. **Sicurezza sul Piano Dati:** Rilevare e mitigare in tempo reale attacchi Man-in-the-Middle (MitM) basati su ARP Spoofing / Cache Poisoning, sfruttando l'intelligenza del controller OpenFlow senza richiedere agenti dedicati sugli host.

2 Ambiente Sperimentale e Prerequisiti Software

Il testbench è stato sviluppato e validato su ambiente Linux (distribuzioni consigliate: **Ubuntu 20.04/22.04 LTS** o appliance Mininet VM). L'infrastruttura richiede l'integrazione di componenti software specifici per la gestione del piano dati, del piano di controllo OpenFlow e dei tool di generazione/ispezione del traffico:

- **Mininet:** framework di virtualizzazione di rete per emulare topologie su namespace Linux isolati.
- **Open vSwitch (OVS):** multilayer switch virtuale programmabile con supporto nativo OpenFlow 1.3 e database OVSDB.
- **Ryu SDN Framework:** framework di controllo in Python utilizzato per l'orchestrazione logica centralizzata e la gestione degli eventi OpenFlow.
- **iperf3:** tool diagnostico per la generazione e misurazione di throughput, jitter e packet loss per flussi UDP/TCP.
- **Scapy:** libreria avanzata di packet manipulation in Python, impiegata nel testbench per forgiare frame ARP malevoli a basso livello.

La predisposizione dell'ambiente e l'installazione delle relative dipendenze su sistema Debian/Ubuntu avviene tramite la seguente sequenza di comandi da terminale:

```
# Aggiornamento dell'indice dei pacchetti di sistema
sudo apt update && sudo apt upgrade -y
```

```
# Installazione dei componenti core, switch OVS e tool di rete
sudo apt install -y mininet openvswitch-switch openvswitch-testcontroller \
    iperf3 iproute2 python3-pip python3-scapy

# Installazione del controller Ryu e dipendenze compatibili
sudo pip3 install ryu eventlet==0.30.2

# Abilitazione e avvio del servizio Open vSwitch
sudo systemctl enable --now openvswitch-switch
```

3 Architettura della Rete e Topologia

La topologia emulata (`topo.py`) riproduce due domini broadcast indipendenti interconnessi a livello di rete da un router software Linux intermedio (`r1`):

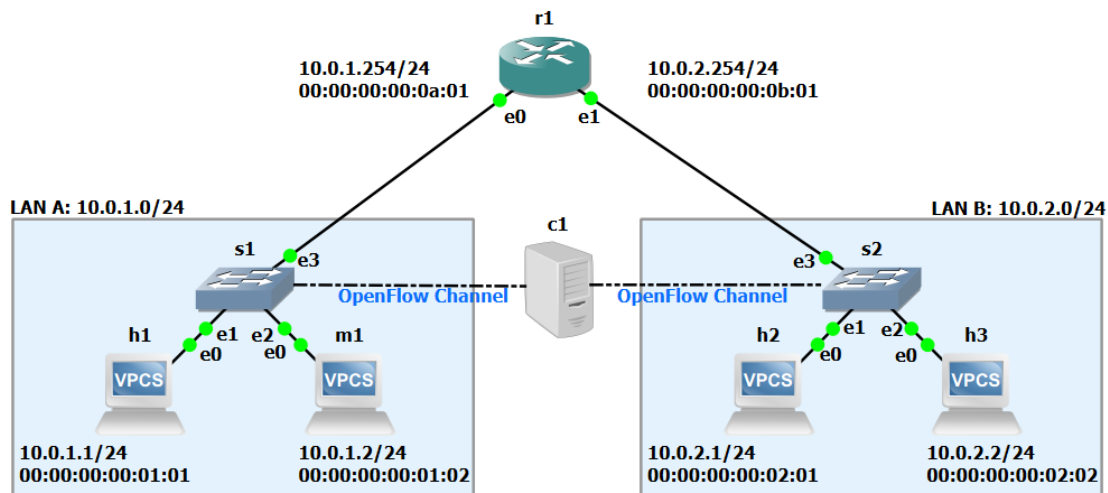


Figura 1: Architettura del testbench SDN: partizione delle subnet e canali OpenFlow verso C1.

- **LAN A (10.0.1.0/24):**
 - h1 (10.0.1.1, MAC 00:00:00:00:01:01): host sorgente legittimo.
 - m1 (10.0.1.2, MAC 00:00:00:00:01:02): host attaccante compromesso.
 - s1: Open vSwitch collegato al gateway r1-eth0 (10.0.1.254) tramite la porta fisica 3 (s1-eth3).
- **LAN B (10.0.2.0/24):**
 - h2 (10.0.2.1, MAC 00:00:00:00:02:01): server destinatario prioritario (porta TCP/UDP 5201).
 - h3 (10.0.2.2, MAC 00:00:00:00:02:02): server destinatario best-effort (porta TCP/UDP 5202).
 - s2: Open vSwitch collegato all'interfaccia r1-eth1 (10.0.2.254).
- **Router L3 (r1):** Nodo Mininet specializzato basato su classe derivata `LinuxRouter` con `net.ipv4.ip_forward = 1`.

4 Ingegnerizzazione del Piano Dati e Configurazione QoS (HTB)

La porta di uscita s1-eth3 costituisce il collo di bottiglia strutturale del sistema. Tramite Open vSwitch Database (**OVSDB**), sull'interfaccia è stato configurato uno scheduler di classe **Hierarchical Token Bucket (HTB)** con capacità complessiva aggregata di 10 Mbps:

- **Coda 0 (Flusso Prioritario per h2):** Banda minima garantita (CIR) pari a 3 Mbps, con banda massima (PIR) estendibile fino a 10 Mbps.
- **Coda 1 (Flusso Best-Effort per h3):** Banda minima garantita (CIR) pari a 7 Mbps, con possibilità di attingere alla banda residua fino a 10 Mbps.

4.1 Mitigazione del Bufferbloat tramite `iproute2`

Per impostazione predefinita, le implementazioni OVS associano alle classi HTB code FIFO con profondità elevata (1000 pacchetti), ritardando lo scarto da congestione e falsando il jitter. Nel progetto è stato applicato un limite rigido di buffer tramite `tc`:

```
tc qdisc change dev s1-eth3 parent 1:1 handle 10: pfifo limit 10
tc qdisc change dev s1-eth3 parent 1:2 handle 20: pfifo limit 10
```

La dimensione fissata a 10 pacchetti induce lo scarto deterministico dell'eccesso non appena il traffico supera la soglia di servizio, garantendo parametri di latenza stringenti.

5 Logica del Piano di Controllo SDN (`controller.py`)

L'applicazione Ryu opera secondo lo standard OpenFlow 1.3 ed è articolata in tre moduli cooperativi:

5.1 A. Routing Reattivo (L2 MAC Learning)

Allo switch features handshake, viene installata una regola di **Table-Miss** (priorità 0) che inoltra i frame non riconosciuti al controller (`OFPACTIONOutput(OFPP_CONTROLLER)`). Quando un pacchetto giunge via `Packet-In`, il controller memorizza la mappatura (`dpid, src_mac`) $\rightarrow$ `in_port` e inietta una regola di flusso a priorità 10 per i successivi inoltri verso `dst_mac`.

5.2 B. Gestione Proattiva della Qualità del Servizio

All'attivazione dello switch d'ingresso `s1` (`dpid = 1`), Ryu installa a priorità 50 le regole dedicate ai flussi UDP inter-subnet:

- Match: `{eth_type = 0x0800, proto = UDP, dst_ip = 10.0.2.1}` $\Rightarrow$ **Actions:** `{SetQueue(0), Output(3)}`
- Match: `{eth_type = 0x0800, proto = UDP, dst_ip = 10.0.2.2}` $\Rightarrow$ **Actions:** `{SetQueue(1), Output(3)}`

Questo meccanismo scavalca la regola di L2 learning standard e assicura l'instradamento nelle code hardware HTB dedicate.

5.3 C. Dynamic ARP Inspection (DAI) e Mitigazione degli Attacchi

- Una regola a priorità 1000 intercetta tutti i pacchetti con `EtherType 0x0806` (ARP) e li forza a salire al controller.
- Ryu confronta i campi `src_ip` e `src_mac` del frame ARP con il dizionario autoritativo interno: `trusted_arp[(dpid, in_port)]`.
- **Se viene rilevata un'incongruenza** (es. l'host `m1` che dichiara l'IP del gateway `10.0.1.254`):
 1. Viene generato un allarme di sicurezza formattato su console e nel file `stats.log`.
 2. Il pacchetto malevolo viene scartato immediatamente (nessuna operazione di `Packet-Out`).
 3. Viene iniettata sullo switch una regola di **DROP** hardware a priorità 100 associata alla porta e al MAC dell'attaccante (`actions = []`), isolando l'host compromesso dal resto della rete.

6 Risultati Sperimentali e Validazione

I test sono stati condotti in modalità automatica headless (`test_script.py` vs `test_noqos.py`).

6.1 Scenario 1: Verifica della QoS sotto Saturazione

È stato generato traffico UDP concorrente da `h1` mediante `iperf3`: un flusso prioritario a 2 Mbps verso `h2` (porta 5201) ed un flusso best-effort saturante a 12 Mbps verso `h3` (porta 5202).

Analisi: Senza QoS, l'eccesso di traffico generato da `h3` riempie l'unico buffer disponibile, scaricando arbitrariamente oltre metà dei pacchetti del flusso prioritario `h2`. Con l'applicazione delle code HTB, Coda 0 isola interamente i 2 Mbps di `h2` all'interno della riserva garantita di 3 Mbps. Il surplus residuo (1 Mbps) viene riallocato alla Coda 1, consentendo a `h3` di attestarsi esattamente sugli 8 Mbps massimi disponibili sul bottleneck senza interferire con `h2`.

Tabella 1: Benchmark Prestazionale: Confronto Architetture con e senza QoS

Parametro	Prestazionale	Rete SENZA QoS (test_noqos.py)	Rete CON QoS HTB (test_script.py)
Throughput h2 (2 Mbps nominali)		~ 1.4 Mbps (degradato)	2.0 Mbps (pieno regime)
Packet Loss h2		~ 51%	0.0% (nessuna perdita)
Jitter Flusso h2		> 15 ms	< 1.2 ms
Throughput (12 Mbps inviati)	h3	~ 8.4 Mbps	~ 8.0 Mbps
Packet Loss h3		~ 30% (scarto non selettivo)	~ 33% (scarto isolato sull'eccesso)

6.2 Scenario 2: Validazione della Mitigazione ARP Spoofing

L'host m1 ha eseguito lo script di attacco test_arp.py inviando frame ARP malevoli:

```
Ether(src="00:00:00:00:01:02")/ARP(op=2, psrc="10.0.1.254",
    hwsrc="00:00:00:00:01:02", pdst="10.0.1.1")
```

- **Esito su Ryu:** Il controller ha intercettato tempestivamente l'anomalia registrando:

```
SECURITY ALERT: ARP Spoofing Detected! DPID: 1, Port: 2,
Expected: 10.0.1.2/00:00:00:00:01:02, Got: 10.0.1.254/00:00:00:00:01:02
```

- **Esito sullo Switch:** Installata la flow entry di DROP hardware. Il successivo tentativo di comunicazione da parte di m1 (ping -c 2 10.0.1.1) ha registrato un **100% packet loss**, dimostrando il blocco hardware e preservando l'integrità della cache ARP di h1.

7 Istruzioni per la Riproducibilità e Testbench Speculare

7.1 Metodologia di Benchmark e Testbench Speculare (No-QoS)

Per garantire un confronto prestazionale rigoroso ed equo a parità di condizioni al contorno, sono stati implementati moduli di controllo e validazione speculari a quelli di produzione:

- **Controller SDN Neutro (controller_noqos.py):** replica esattamente le logiche di L2 learning reattivo e i controlli di sicurezza DAI dell'applicazione principale, omettendo esclusivamente l'installazione proattiva delle azioni SetQueue a priorità 50 verso le code HTB.
- **Topologia Dinamica in test_noqos.py:** poiché la topologia di base creata da topo.py integra direttamente a livello OVSDDB la configurazione delle code HTB su s1-eth3, l'alternativa canonica sarebbe consistita nel dover definire un file statico ridondante (topo_noqos.py). Per mantenere il repository pulito e modulare, lo script test_noqos.py genera autonomamente a runtime una rete gemella priva di vincoli QoS sul bottleneck, evidenziando il reale impatto del traffico concorrente incontrollato rispetto allo scenario ingegnerizzato.

7.2 Esecuzione dei Test

7.2.1 Scenario A: Test della Rete di Produzione (CON QoS HTB)

1. Avvio del Controller SDN (Terminale 1):

```
ryu-manager controller.py
```

2. Esecuzione Automatizzata del Test (Terminale 2):

```
sudo mn -c
mkdir -p results
sudo python3 test_script.py
```

3. **Verifica dei Risultati:** Tutti i report numerici vengono archiviati nella directory `results/`:
- `h2_iperf.txt` e `h3_iperf.txt`: metriche prestazionali di throughput, jitter e packet loss.
 - `arp_attack.txt` e `ping_after_attack.txt`: esito del trigger di attacco e test del blocco.
 - `ovs_queue_stats.txt`: statistiche e contatori delle code OVS.

7.2.2 Scenario B: Test del Benchmark di Controllo (SENZA QoS)

1. Avvio del Controller Neutro (Terminale 1):

```
ryu-manager controller_noqos.py
```

2. Esecuzione del Test sulla Rete Speculare (Terminale 2):

```
sudo mn -c  
sudo python3 test_noqos.py
```